

Reviewer Pack — Minimal Rank of Generalized Matroid Representations over Mon...

Entry e8f0afc59cad · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-25 20:34:41 UTC

Minimal Rank of Generalized Matroid Representations over Monotone k-CLIQUE Circuits

Entry ID: e8f0afc59cad **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-25 20:34:41 UTC

1. Conjecture

Field A (mathematical branch): Enumerative Combinatorics (Generalized Matroids) **Field B** (complexity object): Complexity Theory: Monotone Circuit Complexity for k-CLIQUE

Statement:

[‘For every monotone circuit C computing the k-CLIQUE problem on n variables, there exists a generalized matroid M representing C such that the rank of M is at least $\Omega(n^{1/4})$.’, ‘The rank of a generalized matroid is defined as the minimum number of basis elements needed to represent all elements of the matroid.’, ‘For monotone circuits, the rank of the corresponding generalized matroid is bounded by $O(\log n)$.’]

Rationale (proposer’s reasoning):

[‘Generalized matroids provide a combinatorial framework that can potentially capture the complexity of certain computational problems, including k-CLIQUE.’, ‘The connection between monotone circuits and matroids has been explored before, but this conjecture focuses on the rank of the generalized matroid, which is a less-studied invariant.’, ‘If true, this conjecture would suggest that the difficulty of solving k-CLIQUE can be characterized in terms of matroid theory.’]

Taxonomy category: MONOTONE_CLIQUE (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 9aab8ecd89cccd49

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all monotone circuits C ($n \leq 40$) and their corresponding generalized matroid representations M , the ratio of the computed rank to $n^{1/4}$ exceeds 0.9 AND no circuit has a rank less than $0.5 * n^{1/4}$. It is falsified if there exists at least one circuit with a computed rank of less than $0.5 * n^{1/4}$, or if the ratio of the computed rank to $n^{1/4}$ is less than 0.9 for any circuit.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=0, elapsed=0.0s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_monotone_circuit(n):
        circuit = [False] * n
        for i in range(1, n-1):
            if random.choice([True, False]):
                circuit[i] = True
        return circuit

    def is_monotone(circuit):
        n = len(circuit)
        for i in range(1, n-1):
            if circuit[i] and not (circuit[i-1] or circuit[i+1]):
                return False
        return True

    def construct_matroid_representation(circuit):
        matroid = []
        for i in range(len(circuit)):
            if circuit[i]:
                matroid.append([i])
        return matroid

    def compute_rank(matroid):
        rank = 0
        bases = [set(b) for b in matroid]
        while bases:
            base = bases.pop()
```

```

        rank += 1
        new_bases = []
        for nb in bases:
            if not nb.issubset(base):
                new_bases.append(nb)
        bases = new_bases
    return rank

n = random.randint(5, 40)
circuit = generate_monotone_circuit(n)

if not is_monotone(circuit):
    return {
        "metric_name": "Rank of Generalized Matroid",
        "metric_value": 0.0,
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": "Non-monotone circuit"
    }

matroid = construct_matroid_representation(circuit)
rank = compute_rank(matroid)

return {
    "metric_name": "Rank of Generalized Matroid",
    "metric_value": rank / n**(1/4),
    "instances_tested": 1,
    "conjecture_holds": rank >= 0.5 * n**(1/4) and rank / n**(1/4) > 0.9,
    "counterexample": ""
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_rank = sum(r["metric_value"] for r in results) / len(results)
    std_rank = math.sqrt(sum((r["metric_value"] - mean_rank)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_rank} std={std_rank} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"Non-monotone circuit\" first_failing_seed={first_failing_seed}")
    else:
        print(f"RESULT: INCONCLUSIVE reason=insufficient_evidence")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
'instances_tested': 1, 'conjecture_holds': False, 'counterexample': 'Non-monotone circuit'}
TRIAL: {'metric_name': 'Rank of Generalized Matroid', 'metric_value': 0.0, 'instances_tested': 1, 'co
monotone circuit'}
TRIAL: {'metric_name': 'Rank of Generalized Matroid', 'metric_value': 0.0, 'instances_tested': 1, 'co
monotone circuit'}
TRIAL: {'metric_name': 'Rank of Generalized Matroid', 'metric_value': 0.0, 'instances_tested': 1, 'co
monotone circuit'}
TRIAL: {'metric_name': 'Rank of Generalized Matroid', 'metric_value': 0.0, 'instances_tested': 1, 'co
monotone circuit'}
TRIAL: {'metric_name': 'Rank of Generalized Matroid', 'metric_value': 6.849506782450969, 'instances_
TRIAL: {'metric_name': 'Rank of Generalized Matroid', 'metric_value': 0.0, 'instances_tested': 1, 'co
monotone circuit'}
TRIAL: {'metric_name': 'Rank of Generalized Matroid', 'metric_value': 0.0, 'instances_tested': 1, 'co
monotone circuit'}
TRIAL: {'metric_name': 'Rank of Generalized Matroid', 'metric_value': 0.0, 'instances_tested': 1, 'co
monotone circuit'}
TRIAL: {'metric_name': 'Rank of Generalized Matroid', 'metric_value': 0.0, 'instances_tested': 1, 'co
monotone circuit'}
TRIAL: {'metric_name': 'Rank of Generalized Matroid', 'metric_value': 0.0, 'instances_tested': 1, 'co
monotone circuit'}
TRIAL: {'metric_name': 'Rank of Generalized Matroid', 'metric_value': 0.0, 'instances_tested': 1, 'co
monotone circuit'}
RESULT: FALSIFIED counterexample="Non-monotone circuit" first_failing_seed=23
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

The test has only one instance tested, which is insufficient to draw a conclusion about the conjecture’s validity. The counterexample provided is ‘Non-monotone circuit’, indicating that the test may not be correctly measuring monotone circuits, which is a critical requirement for the conjecture.

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Safety rail: critic_challenge_falsified | original: The test has only one instance tested, which is insufficient to draw a conclusion about the conjecture’s validity. The counterexample provided is ‘Non | next: Investigate why non-monotone circuits were included in the test and ensure that future tests only include monotone circuits. Additionally, conduct further testing with a larger variety of monotone circuits to validate or falsify the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13899
2	preregistration	ollama_remote	glm4:latest	0	0	6444
3	novelty	ollama_remote	glm4:latest	0	0	5104

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
4	novelty	ollama_remote	glm4:latest	0	0	4986
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12626
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11953
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7807
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9178
9	critic	ollama_remote	glm4:latest	0	0	11380
10	judge	ollama_remote	glm4:latest	0	0	6223

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 89600 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in research/audit/e8f0afc59cad.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/e8f0afc59cad.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/e8f0afc59cad.tar.gz (if generated)